

Search

Uninformed search don't use domain-specific knowledge about distance-to-goal, only know goal vs non-goal when test a node.

- 1. **BFS** expands the shallowest unexpanded node first using queue. **Complete:** Yes if branching factor $b < \infty$. **Optimal:** Yes if all step costs are equal. **Time:** $\sum_{d=0}^d b^j = O(b^d)$. **Space:** $O(b^d)$.
- 2. **UCS** expands the node with lowest path cost from root using a PQ ordered by $g(n)$. **Complete:** Yes assuming every step cost > 0 . **Optimal:** Yes. **Time/Space:** $O(b^{1+\lceil C^*/\epsilon \rceil})$, C^* = cost of optimal solution, ϵ = min step cost.
- 3. **DFS** expands the deepest unexpanded node first using a stack. **Complete:** Yes is max depth $m < \infty$. **Optimal:** No. **Time:** $O(b^m)$. **Space:** $O(bm)$. Can get stuck in cycles without repeated-state checking.
- 4. **DLS** is DFS with a depth cutoff l . **Complete:** Yes. **Optimal:** No. **Time:** $O(b^l)$. **Space:** $O(bl)$.
- 5. **IDS** repeatedly runs DLS with increasing l until finds a goal. **Complete:** Yes. **Optimal:** Yes if step cost=1. **Time:** $O(b^d)$. **Space:** $O(bd)$.

Informed search use heuristic knowledge (an estimate of distance-to-goal) to choose which node to expand.

- 1. **Greedy search** uses the heuristic value $h(n)$ as its evaluation through a min-PQ keyed by $h(\text{node})$. **Complete:** Yes in finite spaces. **Optimal:** No. **Time/Space:** $O(b^m)$.
- 2. **A*** always selects for expansion the fringe node with the smallest $f(n) = g(n) + h(n)$ value (combines idea of UCS and Greedy). **Complete/Optimal:** Yes if h is admissible.
- 3. UCS expands nodes by smallest $g(n)$. A* expands nodes by smallest $f(n) = g(n) + h(n) \implies$ if $h(n) = 0$, $f(n) = g(n)$, A* becomes UCS. If every step cost is 1, then $g(n) = \text{depth}(n) \implies$ UCS behaves like BFS. $\text{BFS} \subseteq \text{UCS} \subseteq \text{A}^*$.

Let h^* denote the true min cost, h be estimated cost. Heuristic h is **admissible** if $\forall n, h(n) \leq h^*(n)$.

Let h_1, h_2 be admissible heuristics for the same problem. h_2 **dominates** h_1 if $\forall n, h_2(n) \geq h_1(n)$. **Dominant** heuristic is more informed, and is better for search efficiency.

If we have several admissible heuristics $h_1, \dots, h_m$, and no single one dominates all others, an idea is to define: $h(n) = \max\{h_1(n), \dots, h_m(n)\}$.

A heuristic h is **consistent (monotonic)** if for every parent-child pair (n_p, n_c) : $h(n_p) \leq c(n_p, n_c) + h(n_c)$. If h is consistent, then along any path: $f(n_c) \geq f(n_p)$ for every parent-child pair (n_p, n_c) .

Consistent $\implies$ admissible, converse is not necessarily true.

If h is consistent, then A* is **optimally efficient** among all optimal search algorithms using the same heuristic.

Tie-breaking of g, h : alphabetical / left-to-right if stated.

Hill-Climbing keeps only one current state in memory and repeatedly tries to replace it with a better neighbouring state. Common versions: **steepest ascent**, **steepest descent**. **Pros:** very little memory usage; simple to implement; often finds reasonable solutions in huge spaces where systematic search is impossible. **Cons:** not complete; not optimal; easily gets stuck in local optima; dependence on initial state. **Escaping bad local optima:** random restart, return best result across runs.

Beam search keeps track of k candidate states rather than just one. It can start from one initial state or start from k random states. **Pros:** Memory-efficient as only $O(k)$ states kept. **Cons:** Not complete as might miss goal if pruned early and not optimal since might miss better solutions.

Simulated annealing is like hill climbing, but allows bad moves sometimes to escape local optima. Randomly pick a neighbour and if better move to it, otherwise if worse, move to it with probability p . Repeat until stopping condition met like max iterations or good enough solution. By accepting worse moves early on, it helps explore and escape local traps.

Game Playing

Types: deterministic vs chance; perfect vs imperfect info; zero-sum vs non-zero-sum.

Game as search: state=board, initial=board+turn, operators=legal moves, terminal=game over, utility=terminal payoff, eval=e(s) for non-terminal. Utility/eval from MAX perspective: higher = better for MAX; negative favours MIN.

Minimax: 2-player deterministic perfect-info zero-sum. Assumes opponent optimal. Terminal $\rightarrow U(s)$. MAX $\rightarrow$ max children. MIN $\rightarrow$ min children. Root MAX chooses child with largest backed-up value. If MIN suboptimal, MAX does at least as well as minimax value. **Properties:** DFS, optimal vs optimal opponent, **time** $O(b^m)$, **space** $O(bm)$.

Alpha-Beta: α = best/highest value for MAX so far, init $-\infty$. β = best/lowest value for MIN so far, init $+\infty$. **MAX node:** update α ; if $\alpha \geq \beta \rightarrow$ beta cutoff. **MIN node:** update β ; if $\beta \leq \alpha \rightarrow$ alpha cutoff. DFS / left-to-right unless stated. First branch usually no pruning. Same result as minimax; less/equal evaluated nodes. Worst $O(b^d)$, best $O(b^{d/2})$; good move ordering helps.

CUTOFF + EVAL: Full tree infeasible $\rightarrow$ cut off at fixed depth. Replace TERMINAL-TEST with CUTOFF-TEST. Replace UTILITY with EVAL(s). $\text{EVAL}(s) = \sum_i w_i f_i(s)$.

Horizon effect: Hidden pitfall just beyond depth limit. Apply eval **only** at **quiescent** states. **Non-quiescent:** capture/promotion/king danger; extend search.

Chance/Expectiminimax: Game tree has MAX, MIN, CHANCE nodes. MAX = max; MIN = min, CHANCE = $\sum_i p_i V_i$. Exact values matter in chance games. **Time** $O(b^m n^m)$.

Supervised Learning

KNN

$N_k(\mathbf{x})$ = index set of k nearest training examples to $\mathbf{x}$.

Classification: $\hat{y} = \operatorname{argmax}_{c \in \mathcal{Y}} \sum_{i \in N_k(\mathbf{x})} \mathbf{1}\{y^{(i)} = c\}$.

Regression: $\hat{y} = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y^{(i)}$.

Distance: $L_q: \|\mathbf{x} - \mathbf{x}^{(i)}\|_q = (\sum_{j=1}^d |x_j - x_j^{(i)}|)^{1/q}$. L_1 Manhattan, L_2 Euclidean. For nominal attribute: $\delta(x_j, x_j^{(i)}) = 0$ if same, 1 otherwise (no normalisation needed).

Min-max normalisation:

$$\tilde{x}_j^{(i)} := \frac{x_j^{(i)} - \min\{x_j^{(i)}\}_{i=1}^n}{\max\{x_j^{(i)}\}_{i=1}^n - \min\{x_j^{(i)}\}_{i=1}^n} \in [0, 1]$$

Without scaling, large-scale attributes dominate distance.

Missing values: Missing is maximally different. Nominal: missing diff=1, Numeric normalised: if one missing, diff=max(value, 1-value); if both missing, diff=1

KNN = instance-based/non-parametric/lazy-learning. **Training:** store all training examples. **One prediction** $O(nd)$, **memory** $O(nd)$. **Sensitive** to k , irrelevant attributes, noise, high dimensionality.

Curse of dimensionality: if d is large, distance becomes ineffective metric. **Solution:** feature selection/attribute weighting. Common choice: $k \approx \sqrt{n}$.

1R algorithm

1R = one-rule. Generates one rule testing a **single attribute**. Equivalent to a **1-level decision tree**.

for each attribute X_j do
 for each value $v \in X_j$ do
 $D_{j,v} \leftarrow \{(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{D} : x_j^{(i)} = v\}$
 $\hat{c}_{j,v} \leftarrow \text{majority}\{y^{(i)} : (\mathbf{x}^{(i)}, y^{(i)}) \in D_{j,v}\}$
 end for
 $\text{Err}(X_j) \leftarrow \frac{1}{n} \sum_v |\{(\mathbf{x}^{(i)}, y^{(i)}) \in D_{j,v} : y^{(i)} \neq \hat{c}_{j,v}\}|$
end for
Choose $j^* = \operatorname{argmin}_j \text{Err}(X_j)$
If tie, choose randomly unless question gives tie rule.

Missing values handling: methods

- 1. Ignore examples with missing values.
- 2. Treat missing as another attribute value.
- 3. Replace with most common value of that attribute, or most common value among examples of the same class.

Numeric attributes: Discretise numeric attribute into intervals. Sort examples by attribute value. Place breakpoint when class changes, halfway between adjacent values.

Problem: many intervals $\rightarrow$ overfitting. **Better:** aggregate/merge intervals, require minimum examples per partition.

Naive Bayes

Bayes' theorem:

$$P(y = c | \mathbf{x}) = \frac{P(\mathbf{x} | y = c) P(y = c)}{P(\mathbf{x})}$$

Assume the attributes are **conditional independent** given class; and all attributes **contribute equally**. Assumptions are unrealistic, but works surprisingly well in practice.

Decision rule: $\hat{y} = \operatorname{argmax}_{c \in \mathcal{Y}} P(y = c) \prod_{j=1}^d P(x_j | y = c)$

No need to compute $P(\mathbf{x})$, because denominator is same for all classes. **Estimate** probabilities from training data. Prior: $P(y = c) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y^{(i)} = c\}$

$$\text{Conditional: } P(x_j = v | y = c) = \frac{\sum_{i=1}^n \mathbf{1}\{x_j^{(i)} = v \wedge y^{(i)} = c\}}{\sum_{i=1}^n \mathbf{1}\{y^{(i)} = c\}}$$

Laplace correction: Avoids division by zero.

$$P(x_j = v | y = c) = \frac{1 + \sum_{i=1}^n \mathbf{1}\{x_j^{(i)} = v \wedge y^{(i)} = c\}}{k + \sum_{i=1}^n \mathbf{1}\{y^{(i)} = c\}}$$

where k is number of possible values that x_j takes.

Missing in training example: do not include missing value in the counts; estimate probabilities using only non-missing values.

Gaussian Naive Bayes: Assume **numeric** attribute x_j is Gaussian within each class: $x_j | y = c \sim \mathcal{N}(\mu_{j,c}, \sigma_{j,c}^2)$, where

$$\mu_{j,c} = \frac{1}{n_c} \sum_{i=1}^n x_j^{(i)}, \text{ and } \sigma_{j,c} = \sqrt{\frac{1}{n_c - 1} \sum_{i=1}^n (x_j^{(i)} - \mu_{j,c})^2}$$

are estimated from training examples in class c . Use Gaussian pdf

$$f(x_j | c) = \frac{1}{\sigma_{j,c} \sqrt{2\pi}} \exp \left[-\frac{(x_j - \mu_{j,c})^2}{2\sigma_{j,c}^2} \right]$$

to replace categorical probability: $P(c) \prod_j f(x_j | c)$

Pros: simple, fast, one scan of training data, clear probabilistic meaning, often works well, robust to isolated noise. **Cons:** correlated attributes violate independence assumption. Gaussian assumption for numeric attributes is often false. **Solution:** feature selection, discretisation, other density functions, e.g. binomial, Poisson.

Evaluating and Comparing Classifiers

Training set: used to build classifier. **Validation set:** used to tune parameters / early stopping / pruning. **Test set:** used only for final evaluation. **Rule:** test data must not be used in any way to create/tune classifier.

Holdout: split $\mathcal{D}$ into independent training/test sets, often 2:1. **Repeated holdout:** repeat random train/test split several times, average accuracies.

Stratification: Preserve class proportions in train/test folds. Use with hold-out or cross-validation.

S-fold cross-validation: split $\mathcal{D}$ into S approximately equal non-overlapping folds.

```
Split  $\mathcal{D}$  into  $\mathcal{D}_1, \dots, \mathcal{D}_S$ 
for  $s = 1, \dots, S$  do
  Train on other  $S - 1$  folds
  Test on fold  $\mathcal{D}_s$ 
  Record  $\text{Acc}_s$ 
end for
 $\text{Acc}_{CV} \leftarrow \frac{1}{S} \sum_{s=1}^S \text{Acc}_s$ 
```

$S = 10$ is standard. **Better:** stratified 10-fold CV. Repeated stratified CV reduces random variation.

Leave-one-out CV: train n times, each test set has 1 example. Uses maximum training data but high computational cost.

Precision: among predicted positives, how many are truly positive? **Recall:** among actual positives, how many are found? F_1 : harmonic mean of precision and recall. $P = \frac{TP}{TP + FP}$, $R = \frac{TP}{TP + FN}$, $F_1 = \frac{2PR}{P + R}$.

Comparing classifiers (paired t-test): Define $d_i = \text{Acc}_{1,i} - \text{Acc}_{2,i}$, $\bar{d} = \frac{1}{k} \sum_{i=1}^k d_i$, and $\sigma = \sqrt{\frac{1}{k-1} \sum_{i=1}^k (d_i - \bar{d})^2}$. CI:

$Z = \bar{d} \pm t_{1-\alpha, k-1} \sigma / \sqrt{k}$ contains 0 $\Rightarrow$ difference not statistically significant, otherwise significant.

Cost-sensitive evaluation: different misclassification errors may have different costs. Use weighted/adjusted accuracy, precision, F_1 . Example: false positive and false negative may not be equally bad.

Decision Trees

Non-leaf node tests an attribute; branch = attr val / condition; leaf = predicted class. Prediction: start at root, follow tests until leaf.

ID3/DT construction: Top down recursive D&C.

- 1. Choose best attribute for current node.
 - 2. Split examples by attribute values.
 - 3. Recursively build subtrees.
 - 4. Stop if all examples in subset have same class $\rightarrow$ leaf.
- Stopping criteria:** a) all examples same class $\rightarrow$ leaf with that class. b) can't split further but classes differ $\rightarrow$ leaf with majority class. c) empty subset $\rightarrow$ leaf with majority class of parent subset.

DT = set of mutually exclusive rules. 1 root-to-leaf path = 1 rule. Each rule = conjunction of tests. Tree = disjunction of conjunctions.

$$\text{Entropy: } H(s) = -\sum_{c \in \mathcal{Y}} p_c \log p_c, p_c = \frac{1}{|S|} \sum_{i=1}^{|S|} \mathbf{1}\{y^{(i)} = c\}.$$

Information gain: $G(S, A) = H(S) - H(S|A)$, where $H(S|A) = \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$, and $S_v = \{(\mathbf{x}^{(i)}, y^{(i)}) : x_j^{(i)} = v\}$. G = entropy reduction after splitting on A . Choose attribute with highest G . Higher $G \rightarrow$ purer child nodes.

- Numeric attributes:** Convert to binary split.
- 1. Sort examples by numeric value.
- 2. Candidate split points only between adjacent examples of different labels.
- 3. Split point = halfway between adjacent values.
- 4. Compute G for each candidate split.
- 5. Choose split with highest G .

Pruning avoids overfitting. Pre-pruning: step growing tree early. **Post-pruning:** grow full tree, then prune, more successful/widely used.

Use **validation set** to decide when pruning helps.

Sub-tree replacement: replace subtree by a leaf labelled with majority class of examples reaching that node. Keep pruning if validation accuracy improves or stays same.

Gain ratio: G is biased toward highly-branching attributes may lead to overfitting. Gain ratio penalises highly-branching attributes by considering number and size of branches:

$$\text{SplitInformation}(S, A) = -\sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} \log \frac{|S_v|}{|S|}$$

$$\text{GainRatio}(S, A) = \frac{G(S, A)}{\text{SplitInformation}(S, A)}$$

Decision boundary: axis-parallel/rectangular regions for numeric attributes with binary splits.

Pros: easy to implement, efficient, interpretable. **Cons:** sensitive to noise.

Perceptron

Single output neuron + step activation linear classifier. Can only learn linearly separable data.

$\hat{y} = \text{step}(\mathbf{w}^\top \mathbf{x} + b)$, where $\text{step}(z) = 1$ if $z \geq 0$, 0 otherwise.

Learning rule: For one example, $a^{(i)} = \text{step}(\mathbf{w}^\top \mathbf{x} + b)$, update parameters: $\mathbf{w} \leftarrow \mathbf{w} + \eta(y^{(i)} - a^{(i)})\mathbf{x}^{(i)}$, and $b \leftarrow b + \eta(y^{(i)} - a^{(i)})$, learning rate η is optional.

Perceptron converges in finite steps if data is **linearly separable**.

Multilayer Perceptron

Usually FFN, FC, supervised. Hidden neurons learn intermediate representations. Can model nonlinear decision boundaries through **activation functions** e.g.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \in (0, 1), \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \in (-1, 1).$$

Note: $\sigma'(x) = \sigma(x)(1 - \sigma(x))$, $\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$

Loss: For one example: $\ell = \frac{1}{2} \sum_i (d_i - o_i)^2$, for full training set: $L = \frac{1}{2} \sum_{r=1}^n \sum_i (d_i^{(r)} - o_i^{(r)})^2$

Gradient descent: $\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{\partial L}{\partial \mathbf{w}}$. If η too small $\rightarrow$ slow convergence, if η too large $\rightarrow$ unstable / may diverge. Backprop uses gradient descent, so not guaranteed global minimum.

Backpropagation:

- 1. Forward pass: compute outputs.
- 2. Compute output error signals δ .
- 3. Propagate δ backward to hidden layers.
- 4. Update weights.

For connection from neuron p to q : $w_{pq} \leftarrow w_{pq} + \eta \delta_q o_p$, where for output neuron, $\delta_q = (d_q - o_q) f'(\text{net}_q)$; for hidden neuron: $\delta_q = f'(\text{net}_q) \sum_r \delta_r w_{qr}$.

Batch GD: update after computing total loss over all training examples. **Stochastic GD:** update after each training example. Usually faster and may escape some local optima.

Momentum: $\Delta w_t = \eta \delta_q o_p + \alpha \Delta w_{t-1}$, $w_{t+1} = w_t + \Delta w_t$.

Universal approx. theorems. Any **continuous** function can be approximated with arbitrary small error by a NN with 1 hidden layer. Any function (including discontinuous) can be approximated to arbitrary small error by a NN with 2 hidden layers

Prevent overfitting: validation set, early stopping, smaller network, regularisation, more data. **Early stopping:** stop when validation performance stops improving / validation error increases.

Pros: MLP can model nonlinear decision boundaries. With enough hidden neurons, can approximate complex functions. **Cons:** may find local minimum, not global. Sensitive to initialisation, learning rate, architecture, hidden neurons. Can overfit; needs many examples/computation. Weights are hard to interpret.

Deep Learning

Large NNs that learn useful representations from data. Useful when: data is unstructured, hand-designed features are difficult, data has hierarchical structure. **Problem of traditional FC backprop networks:** slow training, too many parameters, sensitive to initialisation, vanishing/exploding gradients, can get stuck in local minima/plateaus, still may need feature engineering.

Autoencoders

NN trained to reconstruct input: $\mathbf{x}^{(i)} \mapsto \mathbf{h}^{(i)} \mapsto \hat{\mathbf{x}}^{(i)}$. **Goal:** $\hat{\mathbf{x}}^{(i)} \approx \mathbf{x}^{(i)}$. **Encoder:** $\mathbf{h} = f_\theta(\mathbf{x})$. **Decoder:** $\hat{\mathbf{x}} = g_\phi(\mathbf{h})$.

Reconstruction loss: $L_{rec} = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(i)}\|_2^2$. **Main use:** hidden representation $\mathbf{h}$, not output $\hat{\mathbf{x}}$.

Uses: dimensionality reduction, compression, feature extraction, pre-training DNNs.

Stacked autoencoder/pre-training:

- 1. Train AE1: $\mathbf{x} \rightarrow \mathbf{h}_1 \rightarrow \hat{\mathbf{x}}$.
- 2. Keep encoder weights W_1 , discard decoder.
- 3. Train AE2: $\mathbf{h}_1 \rightarrow \mathbf{h}_2 \rightarrow \mathbf{h}_1$.
- 4. Keep W_2 .
- 5. Attach supervised output layer.
- 6. Fine-tune whole network using labelled data + backward.

Why helps: Pre-training gives better initial weights. May converge faster, generalise better, avoid poor local minima, learn meaningful early features. **Other AE:** Sparse (hidden layer may be large, but sparsity penalty forces few active units.), denoising (input is corrupted/noisy $\tilde{\mathbf{x}}$, target is original $\mathbf{x}$. Learns robust features.)

CNN

FC NN for images: need flattening $\rightarrow$ dimensionality too high, too many weights, ignores local spatial structure. **CNN** uses local connectivity, weight sharing, convolutional kernels, pooling $\rightarrow$ good for images because nearby pixels are related.

Local connectivity: each neuron sees only a local patch/receptive field, e.g. $3 \times 3, 5 \times 5$.

Weight sharing: same kernel applied across many image positions. Kernel = learned feature detector. Reduces parameters and is translational equivariant (detects same pattern in different locations).

Convolution operation: slide kernel over image and compute local weighted sums.

$[Y]_{i,j} = \sum_{a=1}^r \sum_{b=1}^s [K]_{a,b} [X]_{i+a-1,j+b-1}$
Output Y is feature map. High feature-map value $\rightarrow$ kernel detects its feature strongly there. Multiple **channels** $\rightarrow$ multiple feature maps, e.g. RGB has 3 channels; kernel extends across channels.

Max Pooling: Given a region $\mathcal{R}$ in image, $\text{MaxPool}(\mathcal{R}) = \max\{z \in \mathcal{R}\}$. Purpose: reduce spatial size, summarise local information, gives local translation robustness/invariance.

Local contrast normalisation: $z'_i := \frac{z_i - \mu_{\text{local}}}{\sigma_{\text{local}}}$. Makes model more robust to brightness changes.

Dropout: Regularisation method to reduce overfitting. Given masks: $\mathbf{m}$, where $m_j = 0$ with probability p , 1 with $1 - p$. The neurons become: $\tilde{\mathbf{h}} = \mathbf{m} \odot \mathbf{h}$. Turned off during testing.

Typical CNN: Conv $\rightarrow$ Activation $\rightarrow$ Pooling $\rightarrow$ optional normalisation, repeat few times, then FC layer(s) $\rightarrow$ output layer.

CNNs are trained by backprop. Shared kernel weight update = sum of gradient contributions from all locations where it was used.

LeNet-5: early CNN for handwritten digit recognition; uses conv + MaxPool + FC layers.

AlexNet: deep CNN success due to conv, ReLU, dropout, GPU, large labelled data.

DL improved because: more computational power, larger datasets, better training techniques, better software. But **DL is not always best:** prefer simpler models when dataset is small, interpretability matters, signal is simple,

training resources are limited, strong baseline already works. Always compare with simple meaningful baselines.

Support Vector Machines

Find decision boundary with max margin. Large margin usually generalises better and less sensitive to noise/overfitting. **Support vectors** = training examples closest to boundary, they define decision boundary.

Linear classifier: $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$. Decision boundary: $\mathbf{w}^\top \mathbf{x} + b = 0$. $\hat{y} = \text{sign}(\mathbf{w}^\top \mathbf{x} + b)$.

Hard margin: $H_1 : \mathbf{w}^\top \mathbf{x} + b = 1$, $H_2 : \mathbf{w}^\top \mathbf{x} + b = -1$. margin = $\frac{1}{\|\mathbf{w}\|}$. Maximise margin $\iff$ find:

$$\underset{\mathbf{w}, b}{\text{argmin}} \frac{1}{2} \|\mathbf{w}\|^2 \text{ s.t. } y^{(i)} (\mathbf{w}^\top \mathbf{x}^{(i)} + b) \geq 1, \forall i = 1, \dots, n$$

requires linearly separable data. **Dual/support vectors:**

$\mathbf{w} = \sum_{i=1}^n \lambda_i y^{(i)} \mathbf{x}^{(i)}$. Many $\lambda_i = 0$. Training examples with $\lambda_i > 0$ are support vectors $\implies$ optimal $\mathbf{w}$ is a linear combination of support vectors. $f(\mathbf{z}) = \sum_{i=1}^n \lambda_i y^{(i)} (\mathbf{x}^{(i)})^\top \mathbf{z} + b$. Predict: $\hat{y} = \text{sign}(f(\mathbf{z}))$. NOTE: during training, inner products are between pairs of training examples $\mathbf{x}^{(i)}, \mathbf{x}^{(j)}$.

Soft margin: allows some misclassification. Useful for noise / nonlinearly separable data. Trades off margin width and training errors. C controls error-margin tradeoff.

Nonlinear SVM: map $\mathbf{x}$ into higher-dimensional space using feature map $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^D$. Find linear max-margin hyperplane in transformed space: $\mathbf{w}^\top \phi(\mathbf{x}) + b = 0$, $\implies$ boundary nonlinear in original space. **Issue:** $\phi(\mathbf{x})$ may be high/infinite-dimensional, hard to compute $\rightarrow$ use kernel trick.

Kernel trick: Kernel function $K : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ computes the inner product in the transformed feature space without explicitly compute the transformed vectors: $K(\mathbf{u}, \mathbf{v}) = \phi(\mathbf{u})^\top \phi(\mathbf{v})$. $\implies$ kernel classification: $f(\mathbf{z}) = \sum_i \lambda_i y^{(i)} K(\mathbf{x}^{(i)}, \mathbf{z}) + b$. Kernel choices: Polynomial: $K(\mathbf{x}, \mathbf{z}) = (\mathbf{x}^\top \mathbf{z} + c)^p$, p - degree, c - constant.

RBF/Gaussian: $K(\mathbf{x}, \mathbf{z}) = \exp\left(-\frac{\|\mathbf{x} - \mathbf{z}\|_2^2}{2\sigma^2}\right)$

Tanh: $K(\mathbf{x}, \mathbf{z}) = \tanh(\mathbf{k}^\top \mathbf{z} - \theta)$ valid only for some parameters.

Ensembles

Combine predictions of multiple base classifiers. **Classification:** majority or weighted vote. **Regression:** average predictions. **Works well** when base classifiers are (1) reasonably accurate, (2) diverse/uncorrelated/don't make same mistakes. (3) If binary base error $\epsilon < 0.5$ and independent, ensemble improves.

$$P(\text{ensemble wrong}) = \sum_{i=0}^M \binom{M}{i} \epsilon^i (1 - \epsilon)^{M-i}$$

Create **diverse** base classifiers by: manipulating training data (bagging, boosting), attributes / features (random forest / subspace), learning algorithm (parameters).

Bagging = bootstrap aggregation.

Require: $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^n$, integer M
for $m = 1, \dots, M$ **do**
 Create bootstrap sample $\mathcal{D}_m$.
 Train base classifier h_m on $\mathcal{D}_m$.
end for

Predict by majority vote; equal classifier weights. Good for unstable classifiers, e.g. DTs.

Boosting: Sequentially build classifiers. Training examples have weights. Misclassified examples get higher weights; correctly classified examples get lower weights. Next classifier focuses on hard examples. Final prediction = weighted vote. Weak learner = slightly better than random, binary error < 0.5 . AdaBoost boosts weak learners into strong ensemble on training data.

Random Forest: bagging + random feature selection for DTs. For each of M trees: bootstrap sample n examples; grow tree without pruning; at each split, consider only k random features; choose best among k , e.g. information gain. Predict by majority vote. Want strong trees with low correlation. As k increases: strength $\uparrow$, correlation $\uparrow$. **Rule of thumb:** $k = \log_2 d$.

Probabilistic Reasoning / Bayesian Networks

RVs $X_1, \dots, X_d$. An assignment is $x_1, \dots, x_d$, where $X_j = x_j$. Event = assignment of one or more variables. Distribution $P(X_j)$ sums to 1. Joint $P(X_1, \dots, X_d)$ gives probability of full assignment. For d Boolean variables, full joint has 2^d entries.

Chain rule:
 $P(\mathbf{X} = \mathbf{x}) = \prod_j P(X_j = x_j | X_1 = x_1, \dots, X_{j-1} = x_{j-1})$

Total probability: $P(A) = \sum_b P(A|B = b)P(B = b)$

Bayes (normalised): $P(A|B) = \frac{\alpha P(B|A)}{P(A)}$

Marginalisation:
 $P(X_j = x_j) = \sum_{\mathbf{x}_{-j}} P(X_1 = x_1, \dots, X_d = x_d)$

Conditional query:

$$\begin{aligned} P(X_q | E = e) &= \alpha P(X_1, E = e) \\ &= \alpha \sum_{\mathbf{h}} P(X_1, E = e, H = \mathbf{h}) \end{aligned}$$

where H are hidden variables. **Conditional independence**, e.g. $A \perp B | C \iff P(A|B, C) = P(A|C)$ is more common than absolute independence. It allows compact probabilistic representation.

Bayesian Network: DAG, node = RV X_j , edge = direct influence/dependence. Each node has conditional probability table: $P(X_j | \text{Pa}(X_j))$, where Pa are the parent RVs. If Boolean X_j has k parents, CPT has 2^k rows. BN size $O(2^d k)$, full joint size $O(2^d)$.

BN joint probability rule:
 $P(\mathbf{X} = \mathbf{x}) = \prod_{j=1}^d P(X_j = x_j | \text{Pa}(X_j))$.

To compute a joint probability:
1. For each node X_j , identify x_j and parent values.
2. Read $P(X_j = x_j | \text{Pa}(X_j))$ from CPT.
3. Multiply all local probabilities.

Example pattern: If $B, E \rightarrow A \rightarrow J, M$, then $P(J, m, a, -b, -e) = P(j|a)P(m|a)P(a|-b, -e)P(-b)P(-e)$.

BN assumption: X_j is conditionally independent of its non-descendants given its parents, i.e. $P(X_j | \text{Pa}(X_j), \text{non-desc}) = P(X_j | \text{Pa}(X_j))$.

Markov blanket of X_j : parents of X_j + children of X_j + parents of children of X_j . X_j is conditionally independent of all other nodes given its Markov blanket.

Enumeration inference: Query variable X_q , evidence $E = e$. Hidden variables H = all variables except X_q and E .

$$P(X_q | e) = \alpha \sum_{\mathbf{h}} \prod_{j=1}^d P(X_j | \text{Pa}(X_j))$$

Burglary-style template:
 $P(B|j, m) = \alpha \sum_e \sum_a P(B)P(e)P(a|B, e)P(j|a)P(m|a)$

Variable Elimination: Avoid repeated computation by storing intermediate factors.

Factor product: multiply factors pointwise over matching assignments.

Sum out variable Z : multiply all factors involving Z , then sum over all values of Z .

- VE steps:**
- 1. Convert CPTs/evidence into factors.
 - 2. Choose elimination order for hidden variables.
 - 3. For each hidden variable Z : multiply factors containing Z ; sum out Z ; add resulting factor back.
 - 4. Multiply remaining factors.
 - 5. Normalise.

Order matters. Heuristic: eliminate variable that minimises next factor size.

Naive Bayes is a special case of BN. Class node Y is root. Feature nodes $X_1, \dots, X_d$ are leaves. Assumption: $X_1, \dots, X_d$ are conditionally independent given Y . $P(Y, \mathbf{X}) = P(Y) \prod_{j=1}^d P(X_j | Y)$
For classification: $P(Y = c | \mathbf{x}) \propto P(Y = c) \prod_j P(X_j = x_j | Y = c)$

Prediction: $\hat{y} = \underset{c \in Y}{\text{argmax}} P(Y = c) \prod_{j=1}^d P(X_j = x_j | Y = c)$

Learning CPT from data by counting:
Root node: $P(Y = c) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}\{y^{(i)} = c\}$.

Feature given class: $P(X_j = v | Y = c) = \frac{\sum_{i=1}^n \mathbf{1}\{x_j^{(i)} = v \wedge y^{(i)} = c\}}{\sum_{i=1}^n \mathbf{1}\{y^{(i)} = c\}}$.

General CPT:

$$P(X_j = v | \text{Pa}(X_j) = \mathbf{u}) = \frac{\sum_{i=1}^n \mathbf{1}\{x_j^{(i)} = v \wedge \text{Pa}(x_j^{(i)}) = \mathbf{u}\}}{\sum_{i=1}^n \mathbf{1}\{\text{Pa}(x_j^{(i)}) = \mathbf{u}\}}$$

Unsupervised Learning

Given $\mathcal{D} = \{\mathbf{x}^{(i)}\}_{i=1}^n$. Goal: discover structure, e.g. clusters. No ground-truth loss; use surrogate objectives, e.g. SSE, silhouette.

Clustering setup

Hard clustering: $f : \{1, \dots, n\} \rightarrow \{1, \dots, k\}$. $K_j = \{i : f(i) = j\}$. $\bigcup_{j=1}^k K_j = \{1, \dots, n\}$, $K_i \cap K_j = \emptyset$, for $i \neq j$.

Goal: High cohesion within clusters, high separation between clusters. **Distances:** Euclidean, Manhattan, Cosine dissimilarity:

$d_{\cos}(\mathbf{x}, \mathbf{z}) = 1 - \frac{\mathbf{x}^\top \mathbf{z}}{\|\mathbf{x}\| \|\mathbf{z}\|}$, distance are scale-sensitive, standardise or min-max normalise features before clustering.

Centroid = mean vector of a cluster: $\mathbf{c}_j = \frac{1}{|K_j|} \sum_{i \in K_j} \mathbf{x}^{(i)}$, need not be a real data point.

Evaluating clustering

Davies-Bouldin: spread / separation. Smaller DB = better clustering.

$$\text{DB} = \frac{1}{k} \sum_{i=1}^k \max_{j \neq i} \frac{\text{spread}(K_i) + \text{spread}(K_j)}{d(\mathbf{c}_i, \mathbf{c}_j)}$$

Silhouette coefficient: For point $\mathbf{x}^{(i)} \in K_j$: $a(i)$ = average distance from $\mathbf{x}^{(i)}$ to points in its own cluster; $b(i) = \min_{m \neq j}$ average distance from $\mathbf{x}^{(i)}$ to points in K_m . $s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$.

$s(i) = 1$: well inside own cluster.
 $s(i) = 0$: on boundary.
 $s(i) < 0$: probably assigned to wrong cluster.
Mean silhouette can help choose k .

k-Means

Input: $\{\mathbf{x}^{(i)}\}_{i=1}^n, k$.
Initialise centroids $\mathbf{c}_1, \dots, \mathbf{c}_k$.
Repeat until centroids/clusters stop changing: for $i = 1, \dots, n$
1. $z^{(i)} \leftarrow \underset{j}{\text{argmin}}_j \|\mathbf{x}^{(i)} - \mathbf{c}_j\|_2^2$. Assign $\mathbf{x}^{(i)}$ to nearest centroid.
2. $\mathbf{c}_j \leftarrow \frac{1}{|K_j|} \sum_{i \in K_j} \mathbf{x}^{(i)}$.

Objective: $J = \sum_{j=1}^k \sum_{i \in K_j} \|\mathbf{x}^{(i)} - \mathbf{c}_j\|_2^2$.

k -means minimises SSE within clusters. Assignment step fixes centroids and minimises J over cluster assignments. Update step fixes assignments and sets centroid to cluster mean. Each step weakly decreases J . Converges to local minimum, not necessarily global. **Complexity:** $O(tknd)$, t #iterations.

Sensitive to initialisation: Different seeds can give different local minima. Mitigations: multiple random restarts, keep lowest J ; k -means++ initialisation.

Limitations: assumes roughly spherical, similar-sized clusters; sensitive to outliers; requires k in advance; works with numeric/vector data; categorical data must be encoded first.

Choosing k : elbow method: plot $J(k)$. choose inflection point; mean silhouette: choose k with highest average silhouette; Davies-Bouldin: choose smallest DB.

Nearest-Neighbour single-pass clustering

Require: threshold t .
 Start first cluster with $\mathbf{x}^{(1)}$.
 for each new $\mathbf{x}^{(i)}$ **do**
 Find closest previous point $\mathbf{x}^{(j)}$ in existing clusters.
 if $d(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \leq t$ **then**
 Add $\mathbf{x}^{(i)}$ to that point's cluster.
 else
 Create new cluster.
 end if
 end for

Uses single-link distance. Order-dependent. One-pass, suitable for streaming/changing data. Small $t \implies$ many clusters. Large $t \implies$ few clusters / one big cluster.

Hierarchical clustering

Produces dendrogram = tree of nested partitions. Cut dendrogram at height h to obtain clusters. Does not need k before running; choose k/h after.

- 1. Start with n singleton clusters.
- 2. Compute pairwise distance matrix.
- 3. Repeat: merge two closest clusters according to linkage, Update cluster distances, record merge height in dendrogram.
- 4. Stop when one cluster remains.

Single link: $d_s(K_a, K_b) = \min\{d(\mathbf{x}, \mathbf{z}) : \mathbf{x} \in K_a, \mathbf{z} \in K_b\}$ = minimum pairwise distance. Can produce chain effect; sensitive to noise.

Complete link: $d_c(K_a, K_b) = \max\{d(\mathbf{x}, \mathbf{z}) : \mathbf{x} \in K_a, \mathbf{z} \in K_b\}$, maximum pairwise distance, produces tighter, compact clusters.

Average link: $d_{\text{avg}}(K_a, K_b) = \frac{1}{|K_a||K_b|} \sum_{\mathbf{x} \in K_a} \sum_{\mathbf{z} \in K_b} d(\mathbf{x}, \mathbf{z})$ = average pairwise distance.

Complexity: space $O(n^2)$ for distance matrix. Naive time $O(n^3)$. Not incremental; adding new point may require rerunning.